

# SFDesign: An R Package for Space-Filling Designs

Shangkun Wang, Weijun Xie and V. Roshan Joseph

**Abstract** Space-filling designs play a crucial role in surrogate modeling, uncertainty quantification, and optimization of computer models. The SFDesign package offers a comprehensive suite of functions to construct various types of space-filling designs, including maximin designs, clustering-based designs, maximin Latin hypercube designs, maximum projection designs, and uniform designs. This paper provides a review of commonly used space-filling designs and highlights the advantages of SFDesign over the existing packages. We demonstrate how SFDesign produces high-quality designs with improved space-filling properties, making it a valuable tool for practitioners to design computer experiments.

## 1 Introduction

Space-filling designs are a class of experimental designs for deterministic computer experiments, which are widely used in science and engineering applications (Joseph, 2026). They ensure that the design points are spread evenly throughout the experimental region, minimizing gaps such that no region is left unexplored.

The motivation behind space-filling designs arises from the need to build accurate surrogate models, such as Gaussian process (or kriging) models for computer simulations (Santner et al., 2018; Gramacy, 2020). A good experimental design should place points strategically in the experimental region to capture variations of the response surface while avoiding unnecessary clustering. There are two general strategies for constructing designs: model-based designs and space-filling designs. Model-based designs, such as the maximum entropy design (Currin et al., 1991) and the integrated mean squared prediction design (Sacks et al., 1989), are often limited in practice due to their dependence on unknown correlation parameters in the Gaussian process model. In contrast, space-filling designs offer a model-free approach, providing a more robust framework for model fitting.

There are several space-filling criteria for assessing the quality of a design (Joseph, 2016). Among them, the minimax and maximin criteria are widely adopted geometric measures, both of which have asymptotic optimality justification for Gaussian process modeling (Johnson et al., 1990). However, in high-dimensional spaces, designs optimized using these criteria often exhibit poor projections in lower-dimensional subspaces. Latin hypercube design (LHD) addresses this issue by constraining that each factor of the design is evenly spaced throughout its range (McKay et al., 1979). It is often combined with the aforementioned minimax or maximin criteria to achieve both good coverage and projection property (Morris and Mitchell, 1995; van Dam, 2008). Maximum projection (MaxPro) designs further improve over maximinLHD, which ensures better projection properties across all subsets of factors. Uniformity is another desirable property for designs, particularly when the objective is integration or uncertainty quantification (Fang et al., 2005).

Several open-source packages are available to construct space-filling designs, especially Latin hypercube designs. The **lhs** package provides functionalities to generate random LHD and options to optimize the design based on various criteria such as maximin, S-optimality, etc. (Carnell, 2024). The **DiceDesign** package provides a comprehensive suite of design choices, including model-based designs via exchange algorithms or stochastic processes, as well as optimized LHDs using genetic algorithms or simulated annealing (Dupuy et al., 2015). **SLHD** package (Ba, 2015) uses simulated annealing to generate maximin LHD, which can also accommodate categorical factors using sliced LHD. The **maximin** package deals with maximin design (Sun and Gramacy, 2024). It improves an existing design by swapping out or optimizing the row with the minimum distance. For minimax designs, the **minimaxdesign** package employs clustering algorithms combined with particle swarm optimization to achieve globally optimal designs (Mak, 2021). For MaxPro design, the **MaxPro** package provides functionality to generate LHDs optimized for the MaxPro criterion, with further refinement made possible through continuous optimization (Ba and Joseph, 2018). In this article, we introduce the **SFDesign** package (Wang and Joseph, 2025) which offers considerable improvements over existing packages for generating maximin designs, MaxPro designs, uniform designs, and minimax designs. We also include a function to generate clustering-based designs, which is not available in other packages. A list of the major functions is presented in Table 1.

| Name              | Description                                        |
|-------------------|----------------------------------------------------|
| maximin.crit      | maximin criteria                                   |
| maxpro.crit       | maximum projection criteria                        |
| uniform.crit      | wrap-around discrepancy                            |
| cluster.error     | clustering error                                   |
| randomLHD         | random LHD                                         |
| maximinLHD        | LHD optimized for maximin criteria                 |
| maxproLHD         | LHD optimized for MaxPro criteria                  |
| uniformLHD        | LHD optimized for wrap-around discrepancy          |
| customLHD         | LHD optimized for a user-defined criteria          |
| clustering.design | designs generated by clustering algorithm          |
| maximin.optim     | a continuous optimizer for maximin-distance design |
| maxpro.optim      | a continuous optimizer for MaxPro design           |
| uniform.optim     | a continuous optimizer for uniform design          |
| continuous.optim  | a continuous optimizer for user-defined criteria   |

Table 1: Major functions of the **SFDesign** package

## 2 Review of space-filling design

In this section, we introduce the notation and formally define various space-filling criteria. Let  $\mathcal{X} \subset \mathbb{R}^p$  denote the experimental region of interest, which is typically assumed to be the unit hypercube  $\mathcal{X} = [0, 1]^p$ . Denote a design of size  $n$  by the design matrix  $D_n = [x_1, \dots, x_n]^T$ , where  $x_i^T$  represents the  $i$ th design point.

A maximin design aims to maximize the minimum pairwise distance between design points, ensuring that points are spread as far apart as possible. It can be obtained by maximizing

$$\phi_{\text{Mm}}(D_n) = \min_{i \neq j} \|x_i - x_j\|_2, \quad (1)$$

where  $\|\cdot\|_2$  denotes the Euclidean norm. On the other hand, minimax designs focus on reducing the largest gap within the design space  $\mathcal{X}$  by minimizing:

$$\phi_{\text{mM}}(D_n) = \max_{x \in \mathcal{X}} \min_i \|x - x_i\|_2.$$

This criterion ensures that the maximum distance from any point in the experimental region  $\mathcal{X}$  to the nearest design point is minimized, effectively reducing the largest uncovered region in the space.

Factor sparsity is commonly observed in real physical systems; that is, only a few factors out of the numerous factors in the system may affect the response. Therefore, in deterministic simulations, we do not want replications when we project points to a lower subspace. A Latin hypercube design (LHD) ensures good one-dimensional projections (McKay et al., 1979). In an LHD, each column of  $D_n$  is a permutation of  $[0.5/n, 1.5/n, \dots, (n-0.5)/n]$ , which guarantees that the projections of the points in each dimension are evenly distributed. For an  $n$ -point design with  $p$  factors, there are  $n^p$  possible LHD configurations; however, not all of them are space-filling. To enhance the space-filling properties of LHDs, we can optimize them with respect to the maximin criterion (Morris and Mitchell, 1995). The optimization is usually done by minimizing a reciprocal distance criterion

$$\phi_{\text{rec}}(D_n) = \left\{ \frac{1}{\binom{n}{2}} \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{1}{\|x_i - x_j\|_2^r} \right\}^{1/r}, \quad (2)$$

which is proportional to  $1/\phi_{\text{Mm}}(D_n)$  for large enough  $r$ .

Although maximin LHDs ensure good one-dimensional and full-dimensional space-filling properties, they may not be good when projected onto other subspaces. To improve the projection properties of a design for all possible subspaces, maximum projection (MaxPro) design was proposed (Joseph et al., 2015). It minimizes :

$$\phi_{\text{maxpro}}(D_n) = \left\{ \frac{1}{\binom{n}{2}} \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{1}{\prod_{l=1}^p (x_{il} - x_{jl})^2} \right\}^{1/p}. \quad (3)$$

Compared to (2), the only difference is that the Euclidean distance in the denominator is replaced by a

“product distance”, but this small change ensures good projections to all subspaces.

Uniformity of the points in the experimental region is another important property of space-filling designs, particularly in the context of mean estimation of a model output. Let  $F_n(x) = 1/n \sum_{i=1}^n I(x_i \leq x)$  be the empirical distribution of the design points, where  $I$  is the indicator function in which the comparison is done component-wise. Let  $f: \mathcal{X} \rightarrow \mathbb{R}$  denote the computer code. In order for the mean estimation error  $|\mathbb{E}(f) - \sum_{i=1}^n f(x_i)/n|$  to be small for *any*  $f$ , it is ideal for  $F_n(x)$  to closely approximate the uniform distribution. Several discrepancies have been proposed, such as star  $L_p$ -discrepancy and modified  $L_p$ -discrepancy, to measure the uniformity of designs (Fang et al., 2000). In this package, we utilize the wrap-around discrepancy because of its computational simplicity and nice projection properties (Hickernell, 1998):

$$\phi_{\text{wa}}(D_n) = -\left(\frac{4}{3}\right)^p + \frac{1}{n^2} \sum_{i,j=1}^n \prod_{k=1}^p \left[ \frac{3}{2} - |x_{ik} - x_{jk}|(1 - |x_{ik} - x_{jk}|) \right]. \quad (4)$$

Finally, we introduce clustering-based designs, which is justified as a way to minimize an upper bound of the integrated prediction error of a Gaussian process. The clustering criterion is:

$$\phi_{\text{cluster}}(D_n) = \sum_{i=1}^n \int_{V_i} \|x - x_i\|^\alpha dx. \quad (5)$$

where  $V_i, i = 1, \dots, n$  are the Voronoi regions. For  $\alpha = 1$ , the criterion is the same as  $K$ -median problem. For  $\alpha$  large enough, the criterion can serve as a surrogate criterion for the minimax design (Mak and Joseph, 2018).

### 3 Optimization algorithms

In this section, we present the optimization algorithms employed in this package and illustrate their effectiveness through comparisons with existing packages.

#### 3.1 Designs based on pairwise distance

Many of the criteria reviewed in Section 2.2, such as maximin criterion, MaxPro, and wrap-around discrepancy, are functions of the pairwise distances among design points. Motivated by this observation, we propose a general optimization framework for improving such criteria, with and without constraints to Latin hypercube designs.

For Latin hypercube designs, the optimization framework consists of two stages: an initial optimization phase using simulated annealing (Algorithm 1, Morris and Mitchell (1995)), followed by a deterministic local search refinement (Algorithm 2). Both stages iteratively enhance the design by swapping coordinates between selected pairs of design points.

In the first stage, design point pairs are randomly selected, and swaps are accepted with a certain probability, even if they do not immediately improve the criterion. This probabilistic acceptance helps prevent the algorithm from getting trapped in local optima, enabling more effective exploration of the design space.

In the second stage, a deterministic local search is performed by systematically scanning the design matrix. Every coordinate of each pair of design points is swapped, but the swap is only accepted if it leads to an improvement in the criterion. This greedy approach efficiently refines the design, converging towards a local optimum.

For designs that are not restricted to Latin hypercubes, the optimization process begins with the optimized LHD obtained through the above process or some initial design provided by the user and then transitions to continuous optimization using the low-storage BFGS algorithm from the NLOpt package (Liu and Nocedal, 1989; Johnson, 2008). If a surrogate objective function is used, such as in maximin designs, an additional simulated annealing step can be applied at the end to directly optimize the original criterion.

We present three specific designs implemented in the packages as follows.

#### Maximin design

Instead of directly optimizing the maximin criterion, we start by minimizing the average reciprocal distance in (2). The advantage of this criterion is that it helps differentiate between designs that have the same maximin measure. The reciprocal distance criterion is equivalent to the maximin criterion for

---

**Algorithm 1** SA: simulated annealing for LHD
 

---

**Require:** Design matrix  $D \in \mathbb{R}^{(n \times p)}$ , criterion  $\phi$ , Initial temperature  $T$ , temperature decay rate  $\lambda$ , Maximum number of iterations  $I_1$ , Maximum number of iterations without improving the global best criterion  $I_2$ .

- 1: Initialize the local and global best criterion value by the current criterion value  $\phi_l = \phi_g = \phi(D)$ .
- 2: Initialize the global best design  $D_g = D$ .
- 3: Initialize a flag variable for the loop  $\text{flag} = \text{True}$ .
- 4: Initialize a counter for total number of iteration  $i = 0$ .
- 5: **while**  $\text{flag}$  **do**
- 6:    $\text{flag} = \text{False}$ .
- 7:   **while**  $i_{\text{nochange}} < I_2$  **do**
- 8:     **if**  $i > I_1$  **then** break.
- 9:     **end if**
- 10:    Randomly select two rows  $l, m$  from  $[1, \dots, n]$  and one column  $k$  from  $[1, \dots, p]$ .
- 11:    Update counter  $i = i + 1$ .
- 12:    Set  $D_{\text{try}} = D$  and swap  $x_{lk}$  and  $x_{mk}$  in  $D_{\text{try}}$ .
- 13:    Compute the criterion  $\phi_{\text{try}} = \phi(D_{\text{try}})$ .
- 14:    **if**  $\phi_{\text{try}} < \phi_g$  **then**
- 15:      $\text{flag} = \text{False}$ .
- 16:      $\phi_g = \phi_{\text{try}}, D_g = D_{\text{try}}$ .
- 17:      $\phi_l = \phi_{\text{try}}, D_l = D_{\text{try}}$ .
- 18:      $i_{\text{nochange}} = 1$ .
- 19:    **else**
- 20:      $i_{\text{nochange}} = i_{\text{nochange}} + 1$ .
- 21:     **if**  $\phi_{\text{try}} < \phi_l$  **then**
- 22:       $\text{flag} = \text{False}$ .
- 23:       $\phi_l = \phi_{\text{try}}, D_l = D_{\text{try}}$ .
- 24:     **else**
- 25:      With probability  $\exp(-(\phi_{\text{try}} - \phi_l)/T)$ ,  $\phi_l = \phi_{\text{try}}, D_l = D_{\text{try}}$ .
- 26:     **end if**
- 27:    **end if**
- 28:   **end while**
- 29:    $T = \lambda T$ .
- 30: **end while**
- 31: **Return:**  $D_g$ .

---

large enough  $r$ . Motivated by the choice in Joseph et al. (2015), we use  $r = 2p$  as the default setting in our package.

The gradient of  $\phi_{\text{rec}}^r$  with each element in  $D_n$  is:

$$\frac{\partial \phi_{\text{rec}}^r}{\partial x_{ik}} = \frac{-2r}{n(n-1)} \sum_{j \neq i} \frac{x_{ik} - x_{jk}}{\|x_i - x_j\|_2^{r+2}}, \quad (6)$$

which can be used for gradient-based continuous optimization of maximin design.

---

**Algorithm 2** Deterministic Swap: deterministic swap for LHD

---

**Require:** Design matrix  $D \in \mathbb{R}^{(n \times p)}$ , criterion  $\phi$ , number of passes of the whole design matrix  $P$ , Maximum number of iterations  $I$ .

```

1: Initialize best criterion value by the current criterion value  $\phi_{\text{best}} = \phi(D)$ .
2: Initialize a counter for total number of iteration  $i = 0$ .
3: for  $j = 1, \dots, P$  do
4:    $\text{flag} = \text{False}$ .
5:   for  $j = 1, \dots, n - 1$  do
6:     for  $m = j, \dots, n$  do
7:       for  $k = 1, \dots, p$  do
8:         if  $i > I$  then break.
9:       end if
10:      Update counter  $i = i + 1$ .
11:      Set  $D_{\text{try}} = D$  and swap  $x_{jk}$  and  $x_{mk}$  in  $D_{\text{try}}$ .
12:      Compute the criterion  $\phi_{\text{try}} = \phi(D_{\text{try}})$ .
13:      if  $\phi_{\text{try}} < \phi_{\text{best}}$  then
14:         $\text{flag} = \text{True}$ .
15:         $\phi_{\text{best}} = \phi_{\text{try}}$ ,  $D = D_{\text{try}}$ .
16:      end if
17:    end for
18:  end for
19: end for
20: if not flag then
21:   Break out of all the loops.
22: end if
23: end for
24: Return:  $D$ .

```

---

The following codes generate a two-dimensional seven-point maximin LHD:

```

R> n <- 7
R> p <- 2
R> D <- maximinLHD(n, p)$design

```

By default, it will go through the two-stage optimization. To perform continuous optimization with a simulated annealing (SA) step at the end, the following code can be used:

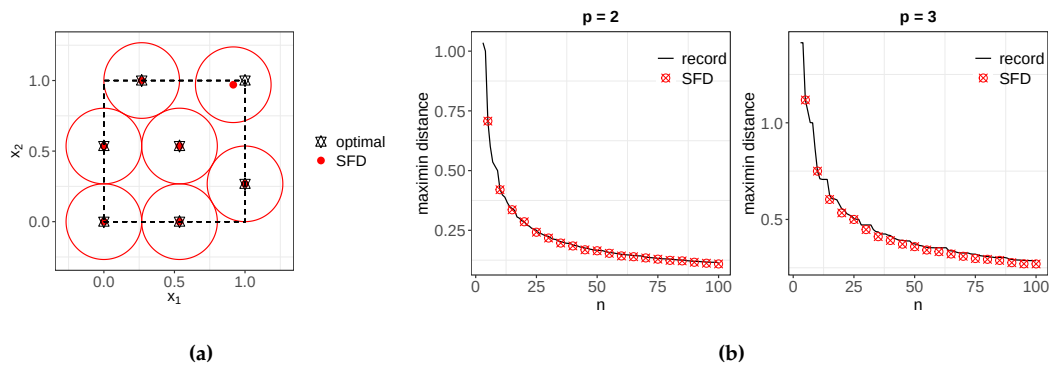
```

R> D <- maximin.optim(D, sa = TRUE)

```

Figure 1a presents the generated seven-point maximin design by the above code (denoted as SFD) alongside with the optimal solution. The design points with the smallest pairwise distance nearly coincide with the optimal points, with the exception of the top-right point, which deviates slightly. This deviation occurs because its placement has minimal impact on the maximin criterion. Maximin designs are closely related to the sphere packing problem (Conway and Sloane, 2013). A comparison of the maximin criterion of the design generated by this package with the best sphere packing solution recorded in the literature (<http://hydra.nat.uni-magdeburg.de/packing/>, denoted as record) is shown in Figure 1b. It can be seen that the designs generated by **SFDesign** are almost optimal. Note that the best sphere packing solutions are available for only two- or three-dimensional problems, whereas our package can be used for any number of dimensions making it useful for computer experiments.

In the above example, we use maximin LHD as the initial design for the `maximin.optim` function. However, if our final goal is to generate a maximin design without LHD constraints, optimizing over



**Figure 1:** (a) Seven-point maximin design in two dimension generated by **SFDesign**. Red circles are centered at the design points and the radius is half the maximin distance. The stars are the optimal design; (b) Comparison of the maximin design for two and three dimension generated by **SFDesign** (denoted as SFD) with the recorded optimal sphere packing solution (denoted as record). The larger, the better.

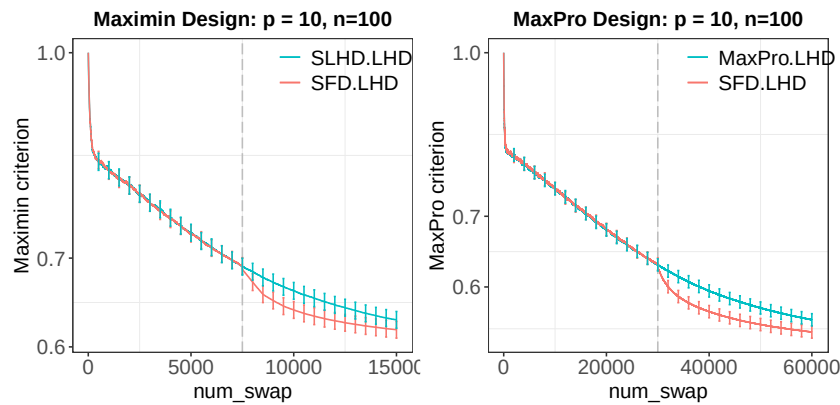
an initial maximin LHD may be inefficient, especially for high dimensions where maximin design points should usually be located at corners. To address this issue, the `maximin.optim` has an option to set `find.best.ini = TRUE`. In this case, in addition to the user-supplied initial design, new initial designs are generated internally for optimization. Given the size of the design  $n$ , the function will first construct the closest full factorial design. If the number of points in this design is less than  $n$ , additional points are added sequentially to augment the design; otherwise, points are removed sequentially to reach the desired size. Initial designs constructed this way typically yield a smaller reciprocal distance criterion after continuous optimization.

We present a comparison with the `maximinSLHD` function from the **SLHD** package (Ba, 2015) and the `maximin` function from the **maximin** package (Sun and Gramacy, 2024) in Figure 4. In the figure, SLHD denotes the maximin LHD generated by **SLHD**; SFD.LHD refers to the LHD obtained after deterministic swap (Algorithm 2); SFD is the result of applying continuous optimization to SFD.LHD, with the setting `find.best.ini = TRUE`, `sa = FALSE`; `maximin` denotes the designs produced by the **maximin** package and it directly minimizes the maximin criterion (Equation 1) instead of the reciprocal distance.

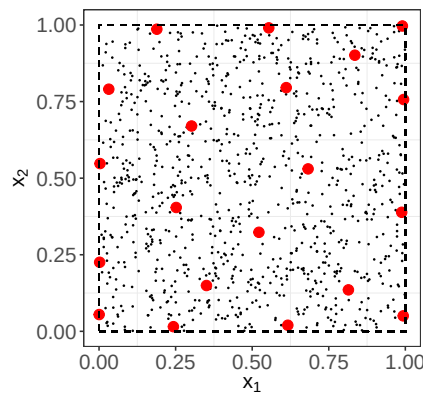
For Latin hypercube designs, a considerable improvement is observed following the deterministic swap given by our package. When the LHD constraint is removed, further enhancement is achieved through continuous optimization, and the result is always better than **maximin**. In higher dimensions (e.g.,  $p = 10$ ), it should be noted that almost all SFD designs came from the internal generation of full factorial-based initial designs rather than the maximin LHD-based initial designs.

We highlight here the efficiency of the deterministic swap algorithm for improving maximin Latin hypercube designs, as illustrated in the left panel of Figure 2. For a fair comparison, the total number of swaps is fixed at 15,000. In our implementation, 7,500 swaps are allocated to the simulated annealing stage and 7,500 to the deterministic swapping stage. Before the onset of deterministic swapping, our method exhibits performance similar to **SLHD**, since both methods employ the same algorithmic framework during the SA stage. Once the algorithm transitions to deterministic swapping, the criterion value decreases faster than **SLHD**.

Besides generating Latin hypercube design and optimizing designs with the continuous optimizer `maximin.optim`, **SFDesign** provides the function `maximin.remove` for users to generate maximin designs from a large pool of candidate points. In Figure 3 we show an example of generating a maximin of size 20 from 1000 candidates.



**Figure 2:** Trajectory of the criterion value with respect to the number of swaps performed by the algorithm. The gray dashed lines indicate when the deterministic swap stage starts. The solid lines represent the mean  $\bar{m}$  over 100 experiments  $\mathbf{m} = (m_1, \dots, m_{100})$ , and the error bars mark  $[\bar{m} - 1.96\text{std}(\mathbf{m})/\sqrt{100}, \bar{m} + 1.96\text{std}(\mathbf{m})/\sqrt{100}]$ . Left: Maximin design, Right: Maximum projection design. SFD represents the SFDdesign package, MaxPro represents the MaxPro package and SLHD represents the SLHD package.



**Figure 3:** A maximin design of size 20 generated by `maximin.remove` from 1000 candidate points. Red points denote the design and the black dots are the candidates.

### Maximum projection design

The major difference of `SFDdesign` and the existing `MaxPro` package (Ba and Joseph, 2018) is the addition of the deterministic swap refinements. Here, we demonstrate how to generate a MaxPro LHD using only the SA step:

```
R> n = 50
R> p = 5
R> D <- maxproLHD(n, p, method='sa')$design
```

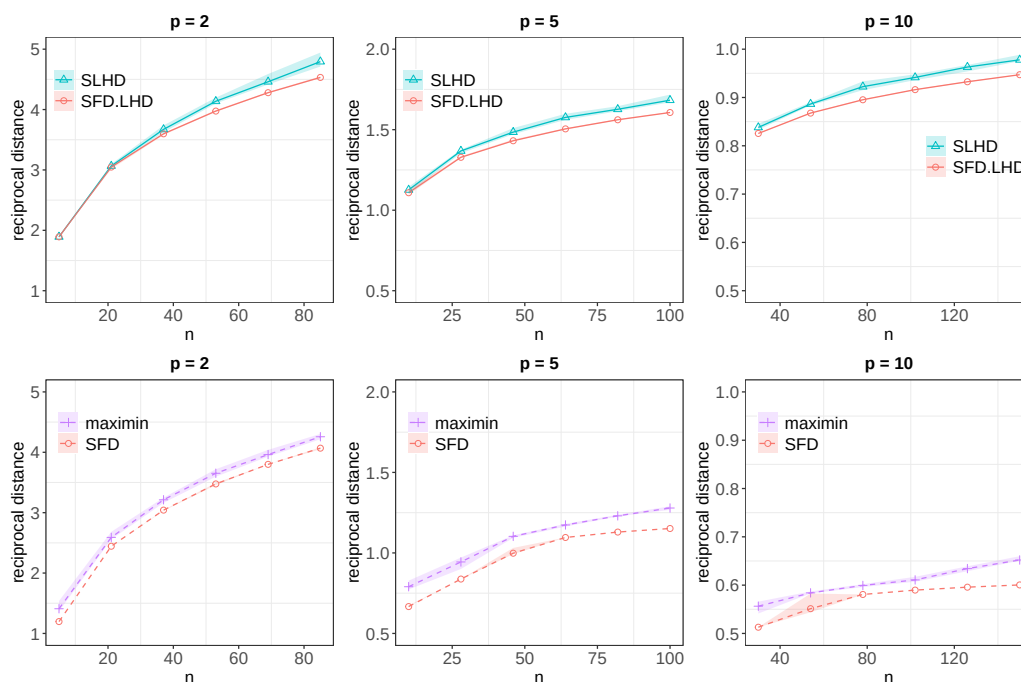
This is equivalent to the `MaxProLHD` function in the `MaxPro` package. However, a deterministic swap of any LHD can be performed by the `SFDdesign` package to improve the MaxPro criterion:

```
R> D <- maxproLHD(D, method='deterministic')$design
```

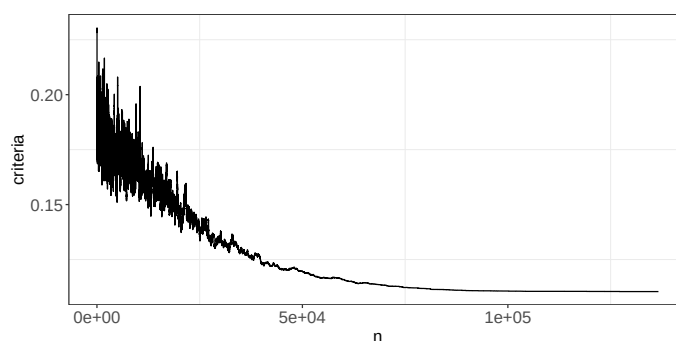
The two steps can be grouped together by the following code:

```
R> result <- maxproLHD(n, p, method='full')
```

The trajectory of the criterion during optimization can be visualized by plotting `result$crit.hist`, as shown in Figure 5. This graph is particularly useful for selecting an appropriate initial temperature for

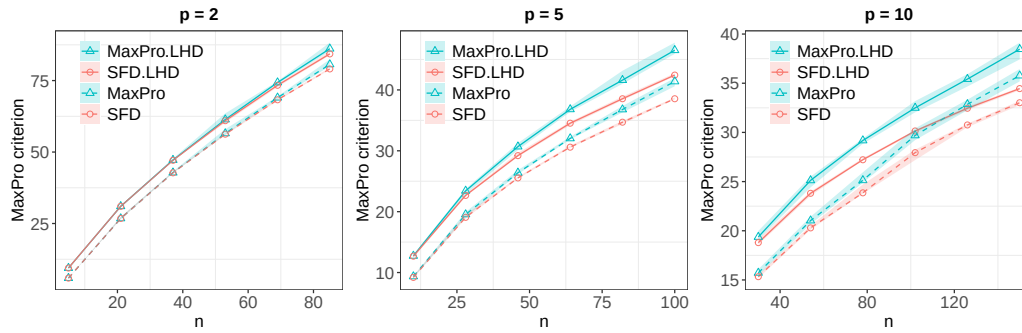


**Figure 4:** Comparison of the reciprocal distance criterion of designs (smaller the better). Top row: SLHD and SFD.LHD denote the maximin LHDs generated by **SLHD** and **SFDesign**, respectively; Bottom row: maximin and SFD denote the maximin designs without LHD constraints generated by **maximin** and **SFDesign**, respectively. Lines represent the median over 10 repetitions and the shaded bands mark the 5th and 95th quantiles. Solid and dashed lines represent designs with and without LHD restrictions, respectively.



**Figure 5:** The trajectory of the MaxPro criterion during optimization.





**Figure 6:** Comparison of the maximum projection criteria (smaller the better) of designs generated by **MaxPro** and **SFDDesign**. MaxPro.LHD and MaxPro are generated by the **MaxPro** package, with and without Latin hypercube (LHD) constraints, respectively. Similarly, SFD.LHD and SFD are designs generated by **SFDDesign** with and without LHD constraints. Lines represent the median over 10 repetitions and the shaded bands mark the 5th and 95th quantiles.

the SA step, especially for user-defined criteria discussed in Section 2.3.1, as their initial temperature is not computed automatically.

A follow-up continuous optimization can be performed to generate a design without LHD constraints:

```
R> D <- maxpro.optim(D)
```

The improvement achieved by deterministic swap is significant, as illustrated in Figure 6. In the figure, solid lines represent results for LHD and dashed lines correspond to designs without LHD constraints. For low-dimensional designs, the improvement is moderate; however, in higher dimensions (e.g.,  $p = 10$ ), the deterministic swap step can, in some cases, enhance the LHD to even outperform designs without LHD constraints generated by the **MaxPro** package.

Similar to the maximin LHD, the deterministic swap algorithm can improve the MaxPro LHD efficiently for the same number of swaps, as shown in the right panel in Figure 2. The total number of swaps is fixed at 60,000 for both **MaxPro** and **SFDDesign**. In our implementation, the number of swaps is fixed at 30,000 in the SA stage and 30,000 in the deterministic swapping stage. Once the algorithm enters the deterministic swapping stage, the MaxPro criterion clearly decreases faster for **SFDDesign**.

## Uniform design

For uniform design, we optimize the wrap-around discrepancy (Equation 4). To generate an LHD with low discrepancy, the following code can be used:

```
R> D <- uniformLHD(n, p)$design
```

For continuous optimization, the gradient of wrap-around discrepancy with respect to each element in  $D_n$  can be derived as follows:

$$\frac{\partial \phi_{wa}}{\partial x_{ik}} = \frac{2}{n^2} \sum_{j \neq i} \prod_{t \neq k} \left[ \frac{3}{2} - |x_{it} - x_{jt}|(1 - |x_{it} - x_{jt}|) \right] \left( -\text{sign}(x_{ik} - x_{jk}) + 2(x_{ik} - x_{jk}) \right). \quad (7)$$

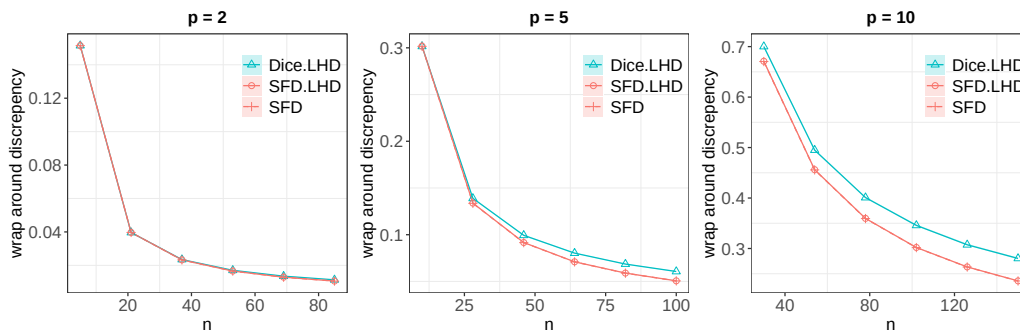
The following code carries out the continuous optimization:

```
R> D <- uniform.optim(D)$design
```

Figure 7 presents a comparison with **DiceDesign**. The improvement is evident, particularly for high-dimensional designs. However, continuous optimization yields only marginal enhancements over the LHD version.

Besides Latin hyper cube design, we provide the functionality to generate a design of discrete factors with different number of levels. For example, suppose that we want to generate a design of size 9 on four factors with levels 3.

```
R> D = uniform.discrete(t = 3, p = 4, levels = rep(3, 4))
```



**Figure 7:** Comparison of the wrap-around discrepancy (smaller the better) of designs generated by **DiceDesign** and **SFDesign**. Dice.LHD denotes uniform LHDs generated by **DiceDesign**, and SFD.LHD and SFD are generated by **SFDesign**, with and without LHD constraint, respectively. Lines represent the median over 10 repetitions and the shaded bands mark the 5th and 95th quantiles.

The generated design is shown in Table 2, which actually is an orthogonal array  $OA(9, 3^4)$  (Wu and Hamada, 2021). Although this need not be the case for larger designs, the generated designs can be close to optimal.

| $x_1$ | $x_2$ | $x_3$ | $x_4$ |
|-------|-------|-------|-------|
| 2     | 3     | 3     | 3     |
| 2     | 1     | 1     | 2     |
| 1     | 3     | 1     | 1     |
| 3     | 2     | 1     | 3     |
| 1     | 2     | 3     | 2     |
| 2     | 2     | 2     | 1     |
| 1     | 1     | 2     | 3     |
| 3     | 1     | 3     | 1     |
| 3     | 3     | 2     | 2     |

**Table 2:** Design generated by uniform.discrete.

### User-defined criteria

The **SFDesign** provides the option to create an LHD based on a user-defined criterion.

```
R> D <- customLHD(compute.distance.matrix, compute.criterion,
  update.distance.matrix, temp, n, p)
```

The user should specify three functions: `compute.distance.matrix` which computes the distance of each pair of design points, `compute.criterion` which computes the user defined criterion from the distance matrix, and `update.distance.matrix` which will update the distance matrix after swapping one column of two rows of the design matrix. If simulated annealing algorithm is used in the optimization, a reasonable initial temperature `temp` should be specified as well so that the algorithm has a balance of exploration and exploitation.

Continuous optimization can also be performed once the user specify the initial design `D.ini` and the objective to minimize and the gradient with respect to the design matrix.

```
R> D <- continuous.optim(D.ini, objective, gradient)
```

### 3.2 Clustering-based design

Clustering-based designs tend to minimize the integrated prediction error of a Gaussian process model. Let  $R(x) = \exp(-\|x\|^2/\theta^2)$  be the Gaussian correlation function and  $r(x) = (R(x - x_1), \dots, R(x - x_n))'$ . Then the root mean squared error for a new point is  $\text{RMSE}(x) = \sqrt{1 - r(x)'R^{-1}r(x)}$ . Let  $\mathcal{X} = V_1 \cup \dots \cup V_n$  be a Voronoi diagram based on the design points. The integrated root mean squared

error (IRMSE) of a design  $D_n$  can be bounded as follows (Joseph, 2026, Ch. 4):

$$\begin{aligned}
 \text{IRMSE}(D_n) &= \int_{\mathcal{X}} \sqrt{1 - \mathbf{r}(x)' \mathbf{R}^{-1} \mathbf{r}(x)} dx \\
 &\leq \sum_{i=1}^n \int_{V_i} \sqrt{1 - R^2(x - x_i)} dx \\
 &= \sum_{i=1}^n \int_{V_i} \sqrt{1 - \exp(-2\|x - x_i\|^2/\theta^2)} dx \\
 &\approx \sum_{i=1}^n \int_{V_i} \sqrt{\|x - x_i\|^2/\theta^2} dx \quad (\text{assume } \|x - x_i\| \ll \theta) \\
 &= \frac{\sqrt{2}}{\theta} \sum_{i=1}^n \int_{V_i} \|x - x_i\| dx.
 \end{aligned}$$

Minimizing this bound is equivalent to the  $K$ -median clustering problem. A generalization of this criteria is the clustering criterion  $\phi_{\text{cluster}}$  in (5). For example, when  $\alpha = 2$ , the clustering criterion is an upper bound for integrated mean square error  $\text{IMSE}(D_n) = \int_{\mathcal{X}} \{1 - \mathbf{r}(x)' \mathbf{R}^{-1} \mathbf{r}(x)\} dx$ . This package provides the option to generate designs based on the clustering criterion for any  $\alpha > 0$ , which is a functionality not provided by the existing packages. The design space is approximated using a large sample of Sobol points,  $S$ . We implement Lloyd's algorithm to generate cluster centers as design points, which iteratively refines an initial set of cluster centers by alternating between partitioning the design space into new Voronoi regions and updating the centers accordingly.

For  $\alpha \leq 2$ , we update each center by Weiszfeld's algorithm. Let  $x_i^{(0)} = x_i$  denote the initial position of the  $i$ th center and let  $S_i$  represent the points within its Voronoi cell. The center is then updated as:

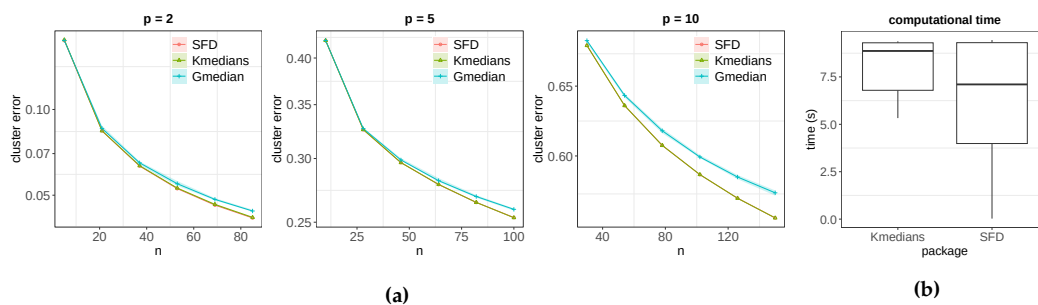
$$x_i^{(k+1)} = \left( \sum_{s \in S_i} \frac{s}{\|s - x_i^{(k)}\|_2^{2-\alpha}} \right) / \left( \sum_{s \in S_i} \frac{1}{\|s - x_i^{(k)}\|_2^{2-\alpha}} \right) \quad \text{for } k = 0, 1, \dots$$

For  $\alpha > 2$ , the centers are updated by accelerated gradient descent, similar to computing  $C_q$ -center in Mak and Joseph (2018).

Here, we mainly focus on  $\alpha = 1$ , which corresponds to minimizing the IRMSE.

```
R> D <- cluster.based.design(n, p, alpha=1)
```

Figure 8a shows a comparison with **Kmedians** (Godichon-Baggioni and Surendran, 2023) and **Gmedian** (Cardot, 2022). Since **SFDesign** employs the same algorithm as **Kmedians**, both exhibit similar performance, and both outperform **Gmedian**. However, the C++ implementation makes **SFDesign** more efficient, as shown in Figure 8b. Moreover, **SFDesign** enjoys the flexibility of different  $\alpha$ . For example, choosing  $\alpha = 10$  can lead to an approximate minimax design. The **Gmedian** package, which implements an averaged stochastic gradient algorithm, is designed to handle large samples in high-dimensional spaces efficiently (Cardot et al., 2012). When the design dimension is extremely high, using **Gmedian** may be advantageous.



**Figure 8:** (a) Comparison of the clustering error of designs generated by **Gmedian**, **Kmedians** and **SFDesign** (denoted as SFD). The smaller the better. Lines represent the median over 10 repetitions and the shaded bands mark the 5th and 95th quantiles; (b) Comparison of computational time for generating fifty-point designs of 5 dimension. Boxplots are generated from 20 experiments.

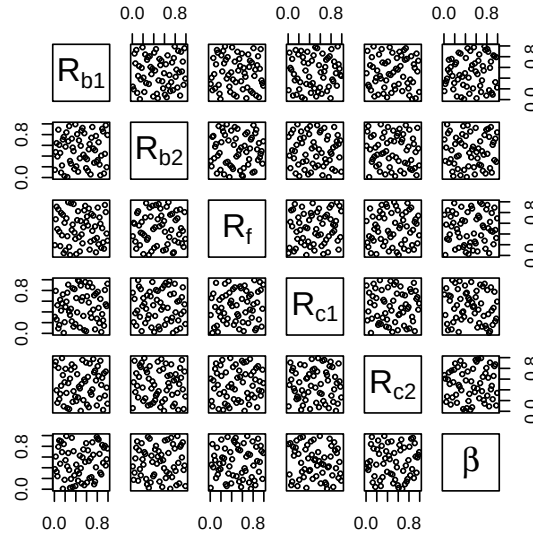


Figure 9: Maximum projection LHD generated by SFDesign

## 4 Case studies

### 4.1 Surrogate modeling

Here we consider the computer model for computing the midpoint voltage  $V_m$  of an output transformerless (OTL) push-pull circuit (Ben-Ari and Steinberg, 2007). There are six input variables: resistance b1 (K-Ohms)  $R_{b1}$ , resistance b2 (K-Ohms)  $R_{b2}$ , resistance f (K-Ohms)  $R_f$ , resistance c1 (K-Ohms)  $R_{c1}$ , resistance c2 (K-Ohms)  $R_{c2}$  and current gain (Amperes)  $\beta$ .

$$V_m(\mathbf{x}) = \frac{(V_{b1} + 0.74)\beta(R_{c2} + 9)}{\beta(R_{c2} + 9) + R_f} + \frac{11.35R_f}{\beta(R_{c2} + 9) + R_f} + \frac{0.74R_f\beta(R_{c2} + 9)}{(\beta(R_{c2} + 9) + R_f)R_{c1}}, \quad \text{where}$$

$$V_{b1} = \frac{12R_{b2}}{R_{b1} + R_{b2}}$$

Four inert input variables are added in order to show the importance of projection properties of designs. We tried two types of design with a design size of fifty: maximin LHD and MaxPro LHD. Figure 9 shows the six active variables (scaled to the unique hypercube) of MaxPro LHD generated by SFDesign.

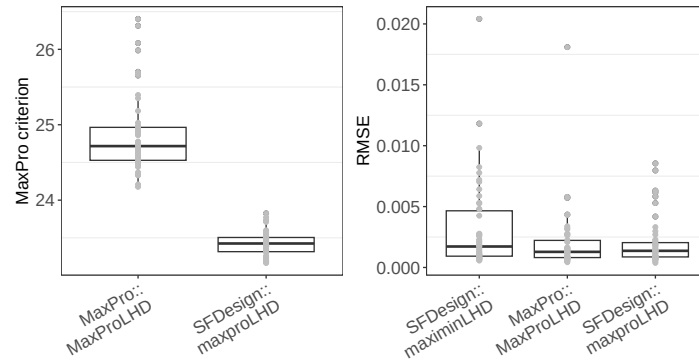
Given the design and corresponding responses, we fit an ordinary kriging model using the **rkriging** package (Huang and Joseph, 2024). To evaluate the model's performance, we generate  $10^6$  Sobol points as the test dataset. The left panel of Figure 10 compares the MaxPro criterion for designs generated by MaxPro and SFDesign. Consistent with Figure 6, SFDesign demonstrates a notable improvement over MaxPro. Additionally, we compare the prediction root mean square error (RMSE) of the kriging models. The results indicate that ordinary kriging models fitted on MaxPro LHDs achieve lower prediction errors, with SFDesign providing more consistent improvements.

### 4.2 Uncertainty quantification

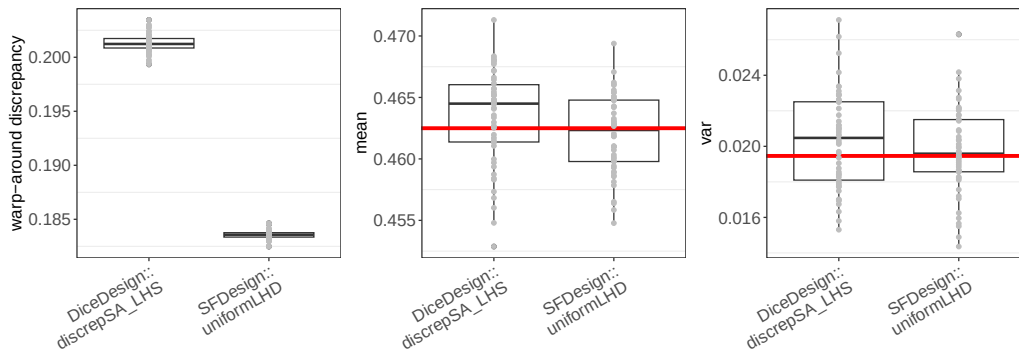
When the objective is to quantify the uncertainty of a computer code, uniform designs are desired. Here we consider a simulation of the piston motion within a cylinder (Kenett and Zacks, 1998). The cycle time in seconds is given by

$$C(\mathbf{x}) = 2\pi \sqrt{\frac{M}{k + S^2 \frac{P_0 V_0}{T_0} \frac{T_a}{V^2}}}, \quad \text{where}$$

$$V = \frac{S}{2k} \left( \sqrt{A^2 + 4k \frac{P_0 V_0}{T_0} T_a} - A \right)$$



**Figure 10:** Left: Boxplots of maximum projection criterion of maximum projection LHDs generated by MaxPro and SFDesign. Right: Prediction RMSE of ordinary kriging fitted on maximin LHD designs generated by SFDesign and maximum projection LHDs generated by MaxPro and SFDesign. The result is generated from 20 random experiments.



**Figure 11:** Boxplots of wrap-around discrepancy of designs generated by functions from **MaxPro** and **SFDesign**, and the mean and variance estimation based on the corresponding experimental responses. The red line is the true mean and variance of the response. The result is generated from 20 random experiments.

$$A = P_0 S + 19.62 M - \frac{k V_0}{S}$$

where the piston weight  $M \in [30, 60]$  kg, the piston surface area  $S \in [0.005, 0.020]$  m<sup>2</sup>, initial gas volume  $V_0 \in [0.002, 0.010]$  m<sup>3</sup>, spring coefficient  $k \in [1000, 5000]$  N/m, the atmospheric pressure  $P_0 \in [90, 000, 110, 000]$  N/m<sup>2</sup>, ambient temperature  $T_a \in [290, 296]$  K, and filling gas temperature  $T_0 \in [340, 360]$  K.

We are interested in computing the mean and variance of the cycle time when the inputs are uniformly distributed within their respective ranges. Figure 11 shows that the wrap-around discrepancy of uniform designs generated by SFDesign is lower, leading to more accurate mean and variance estimations that are closer to the true values.

## 5 Conclusion

In this paper, we introduce the **SFDesign** package, a comprehensive set of functions for constructing space-filling designs based on several widely used criteria. Through a combination of algorithmic advancements and software optimizations, our package achieves substantial improvements over existing packages, offering more efficient and effective design generation. In addition to supporting established space-filling criteria such as maximin, MaxPro, and uniformity-based discrepancy measures, the package provides a framework for incorporating customized criteria, enabling researchers and practitioners to adapt the design methodology to domain-specific applications. Future extensions may include support for additional criteria beyond those based on pairwise distance or clustering, expanding the scope of design optimization to accommodate alternative space-filling measures.

## Acknowledgments

This work is supported by U.S. National Science Foundation grant DMS-2310637.

## References

- S. Ba. *SLHD: Maximin-Distance (Sliced) Latin Hypercube Designs*, 2015. R package version 2.1-1. available at <https://CRAN.R-project.org/package=SLHD>. [p123, 128]
- S. Ba and V. R. Joseph. *MaxPro: Maximum Projection Designs*, 2018. R package version 4.1-2. available at <https://CRAN.R-project.org/package=MaxPro>. [p123, 129]
- E. N. Ben-Ari and D. M. Steinberg. Modeling data from computer experiments: an empirical comparison of kriging with MARS and projection pursuit regression. *Quality Engineering*, 19(4):327–338, 2007. [p134]
- H. Cardot. *Gmedian: Geometric Median, k-Medians Clustering and Robust Median PCA*, 2022. R package version 1.2.7. available at <https://CRAN.R-project.org/package=Gmedian>. [p133]
- H. Cardot, P. Cénac, and J.-M. Monnez. A fast and recursive algorithm for clustering large datasets with k-medians. *Computational Statistics & Data Analysis*, 56(6):1434–1449, 2012. [p133]
- R. Carnell. *lhs: Latin Hypercube Samples*, 2024. R package version 1.2.0. available at <https://CRAN.R-project.org/package=lhs>. [p123]
- J. H. Conway and N. J. A. Sloane. *Sphere packings, lattices and groups*, volume 290. Springer Science & Business Media, 2013. [p127]
- C. Currin, T. Mitchell, M. Morris, and D. Ylvisaker. Bayesian prediction of deterministic functions, with applications to the design and analysis of computer experiments. *Journal of the American Statistical Association*, 86(416):953–963, 1991. [p123]
- D. Dupuy, C. Helbert, and J. Franco. Dicedesign and diceeval: Two R packages for design and analysis of computer experiments. *Journal of Statistical Software*, 65:1–38, 2015. [p123]
- K.-T. Fang, D. K. Lin, P. Winker, and Y. Zhang. Uniform design: theory and application. *Technometrics*, 42(3):237–248, 2000. [p125]
- K.-T. Fang, R. Li, and A. Sudjianto. *Design and modeling for computer experiments*. Chapman and Hall/CRC, 2005. [p123]
- A. Godichon-Baggioni and S. Surendran. *Kmedians: K-Medians*, 2023. R package version 2.2.0. available at <https://CRAN.R-project.org/package=Kmedians>. [p133]
- R. B. Gramacy. *Surrogates: Gaussian process modeling, design, and optimization for the applied sciences*. Chapman and Hall/CRC, 2020. [p123]
- F. Hickernell. A generalized discrepancy and quadrature error bound. *Mathematics of computation*, 67(221):299–322, 1998. [p125]
- C. Huang and V. R. Joseph. *rkriging: Kriging Modeling*, 2024. R package version 1.0. available at <https://CRAN.R-project.org/package=rkriging>. [p134]
- M. E. Johnson, L. M. Moore, and D. Ylvisaker. Minimax and maximin distance designs. *Journal of statistical planning and inference*, 26(2):131–148, 1990. [p123]
- S. G. Johnson. *The NLOpt nonlinear-optimization package*, 2008. available at <https://github.com/stevengj/nlopt>. [p125]
- V. R. Joseph. Space-filling designs for computer experiments: A review (with discussions). *Quality Engineering*, 28(1):28–44, 2016. [p123]
- V. R. Joseph. *Experimental design for data science and engineering*. Chapman & Hall/CRC Press, ISBN: 9781041117520, 2026. [p123, 133]
- V. R. Joseph, E. Gul, and S. Ba. Maximum projection designs for computer experiments. *Biometrika*, 102(2):371–380, 2015. [p124, 127]
- R. Kenett and S. Zacks. *Modern industrial statistics: design and control of quality and reliability*. Duxbury Press, Pacific Grove, CA, 1998. [p134]

- D. C. Liu and J. Nocedal. On the limited memory BFGS method for large scale optimization. *Mathematical programming*, 45(1):503–528, 1989. [p125]
- S. Mak. *minimaxdesign: Minimax and Minimax Projection Designs*, 2021. URL <https://CRAN.R-project.org/package=minimaxdesign>. R package version 0.1.5. [p123]
- S. Mak and V. R. Joseph. Minimax and minimax projection designs using clustering. *Journal of Computational and Graphical Statistics*, 27(1):166–178, 2018. [p125, 133]
- M. D. McKay, R. J. Beckman, and W. J. Conover. A comparison of three methods for selecting values of input variables in the analysis of output from a computer code. *Technometrics*, 21(2):239–245, 1979. [p123, 124]
- M. D. Morris and T. J. Mitchell. Exploratory designs for computational experiments. *Journal of statistical planning and inference*, 43(3):381–402, 1995. [p123, 124, 125]
- J. Sacks, S. B. Schiller, and W. J. Welch. Designs for computer experiments. *Technometrics*, 31(1):41–47, 1989. [p123]
- T. J. Santner, B. J. Williams, and W. I. Notz. *The Design and Analysis of Computer Experiments*. Springer, New York, NY, 2018. [p123]
- F. Sun and R. B. Gramacy. *maximin: Space-Filling Design under Maximin Distance*, 2024. R package version 1.0-5. available at <https://CRAN.R-project.org/package=maximin>. [p123, 128]
- E. R. van Dam. Two-dimensional minimax Latin hypercube designs. *Discrete Applied Mathematics*, 156(18):3483–3493, 2008. [p123]
- S. Wang and V. R. Joseph. *SFDesign: Space-Filling Designs*, 2025. R package version 0.1.0. [p123]
- C. F. J. Wu and M. S. Hamada. *Experiments: planning, analysis, and optimization*. John Wiley & Sons, 2021. [p132]

Shangkun Wang  
H. Milton Stewart School of Industrial and Systems Engineering,  
Georgia Institute of Technology  
Atlanta, GA 30332  
USA  
[shangkunwang01@gmail.com](mailto:shangkunwang01@gmail.com)

Weijun Xie  
H. Milton Stewart School of Industrial and Systems Engineering,  
Georgia Institute of Technology  
Atlanta, GA 30332  
USA  
[wxie@gatech.edu](mailto:wxie@gatech.edu)

V. Roshan Joseph  
H. Milton Stewart School of Industrial and Systems Engineering,  
Georgia Institute of Technology  
Atlanta, GA 30332  
USA  
[roshan@gatech.edu](mailto:roshan@gatech.edu)